PROGRAMMING WITH DATA

WEATTHER CHATBOT [REPORT]

Table of Contents

Week-by-Week Iterative Development and Feature Enhancements 2

Week 1–4: Part 1 2

Basic Rule-Based Chatbot with Lists and Conditions 2

Feedback & Reflection: 2

Improvements: 2

Week 5–6: Part 2 2

Dictionaries and Regular Expression Pattern Matching 2

Feedback & Reflection: 2

Improvements: 2

Week 7–8: Part 3 3

Data-Driven Chatbot with JSON Integration 3

Feedback & Reflection: 3

Improvements: 3

Chatbot Development Report – Part 4: Weather Domain Chatbot 3

Application Overview 3

Key Functionalities 3

Dataset and Intent Structure 4

Test Cases 4

Personalized Interaction Features 5

Code Organization and Structure 5

Advanced Features Used 5

Week-by-Week Iterative Development and Feature Enhancements

Week 1–4: Part 1

Basic Rule-Based Chatbot with Lists and Conditions

In the initial stage, the chatbot was built using simple Python constructs: lists, conditional if-else statements, and basic string matching. The chatbot could identify keywords like “hello,” “tea,” and “thank you” and respond with predefined responses stored in parallel lists (precepts and responses). The chatbot looped continuously, terminating only when the user typed “bye”.

Feedback & Reflection:

The feedback emphasized limited flexibility—responses were overly rigid and failed to accommodate variations in input phrasing (e.g., “Hi” vs “Hello there”). The need for a more dynamic response mechanism was identified.

Improvements:

- Used list indexing to match inputs with appropriate responses.
- Added a termination message and graceful exit logic.

Week 5–6: Part 2

Dictionaries and Regular Expression Pattern Matching

This phase introduced **dictionaries for mapping** inputs to responses and **regular expressions (RE)** for pattern recognition. Multiple patterns like greetings, farewells, gratitude, and queries about the bot’s name or feelings were grouped under REs using the re module. Advanced REs were also created for extracting **user name and favorite color**, enhancing personalization.

Feedback & Reflection:

Feedback pointed out the growing need for **modularity** and **code reuse**. There was also a recommendation to handle more **complex and flexible patterns** (e.g., “You can call me Alice”).

Improvements:

- Modularized functions: detect_pattern(), chatbot_response(), re_patterns(), and re_responses() were developed for maintainability.
- Captured dynamic user attributes (name and color) using RE groups and stored them in a shared user_state dictionary.
- Used format() strings for dynamic, context-aware responses.
- Added error handling and fallback responses for unmatched inputs.

Week 7–8: Part 3

Data-Driven Chatbot with JSON Integration

This final stage focused on **scaling up** the chatbot by integrating a JSON file (intents.json) containing categorized intents, patterns, and responses. The chatbot dynamically loaded the file, generated RE-to-intent (rgx2int) and intent-to-response (int2res) dictionaries, and responded based on matched patterns.

Feedback & Reflection:

The recommendation was to transition toward a more **data-driven design** to improve scalability and flexibility. A structured approach was needed to manage more diverse interactions without hardcoding logic into the Python file.

Improvements:

- Implemented load_files() to read and structure intents and responses from JSON.
- Developed generate_response() to dynamically match user input and return a randomized appropriate response.
- Used random.choice() for variety in responses.

Chatbot Development Report – Part 4: Weather Domain Chatbot

Application Overview

This report documents the development of a rule-based chatbot that operates in the **weather information domain**. The chatbot is designed to respond to user queries regarding current weather conditions, forecasts, temperature, seasonal variations, humidity, air quality, UV index, and weather safety tips. It also includes personalized interaction through user-specific name, location, and timezone settings.

The chatbot supports natural language interaction using pattern matching and responds with personalized replies enriched with relevant emojis for improved user engagement. It simulates location-aware behavior using placeholder replacement and provides time-sensitive greetings based on the user's timezone.

Key Functionalities

The chatbot performs the following functions:

- **Personalized greetings** based on time of day and user's timezone (e.g., "Good evening 🌙").
- **Location-specific responses** using a {location} placeholder in all relevant responses.
- **Contextual responses** across 13 unique intents including:

    - Greeting
    - Goodbye
    - Current Weather
    - Weather Forecast
    - Temperature Query
    - Seasonal Weather
    - Extreme Weather
    - Weather Safety
    - Climate vs Weather
    - UV Index
    - Humidity Conditions
    - Air Quality
- **Natural language preprocessing** using NLTK: tokenization, stopword removal, stemming, and lemmatization.

Dataset and Intent Structure

The chatbot uses a structured JSON file to define its conversational logic through a series of intents. Each intent consists of a tag (e.g., "greeting", "current_weather"), a set of patterns representing possible user queries, and corresponding responses that the bot selects from randomly.

To ensure varied and natural interactions, each intent includes multiple phrasings of similar questions, such as “What’s the weather like today?” or “Is it sunny outside?” under the "current_weather" tag. Responses contain dynamic placeholders like {location} and {name}, which are replaced at runtime based on user input, allowing for a personalized experience.

The dataset includes 13 well-defined intents, covering topics like general greetings, current weather, forecasts, temperature, air quality, UV index, and weather safety. This structure makes the chatbot scalable, easy to maintain, and effective within the domain of weather-based interactions.

Test Cases

Test Case	User Input	Bot Response
Greeting + Timezone	Hello → Asia/Karachi → Ali	Good evening 🌙! What's your name? then Nice to meet you, Ali from Lahore! 😊
Weather Query	What's the weather like today?	In Lahore, it depends on your exact area 🌍. Check your weather app for accurate info!
Air Quality	Is the pollution bad?	High pollution in Lahore? Wear a mask 😷

Personalized Interaction Features

The chatbot enhances user experience through three key personalization strategies:

- **Name Integration**: The user's name is requested at the start and dynamically inserted into responses using the {name} placeholder, creating a more conversational tone.
- **Location Awareness**: Users are asked to provide their location, which is used in responses via the {location} placeholder. Though not API-driven, it simulates geo-personalization.
- **Timezone-Based Greetings**: Using pytz and datetime, the chatbot detects the user's local time and offers appropriate greetings:
  - Morning: "Good morning ☀️"
  - Afternoon: "Good afternoon 🌤️"
  - Evening: "Good evening 🌙"

These features make interactions feel tailored and human-like.

Code Organization and Structure

The chatbot is modularly structured and written in Python using best practices for readability and maintainability. Key components include:

- **chatbot()**: The main interactive loop that manages greeting, input prompts, exit logic, and user flow. It handles user profile setup (name, location, timezone) and continuously listens for user input until the termination command ("bye").

- **generate_response(user_input, intents_json)**: This function handles preprocessing of user input, compares it to preprocessed patterns in the intents.json file, and returns an appropriate response. It also replaces {name} and {location} placeholders with actual user data.
- **preprocess_text(text, debug=False)**: A preprocessing pipeline using NLTK that performs:
  - Lowercasing
  - Punctuation removal
  - Tokenization (word_tokenize)
  - Stopword removal (nltk.corpus.stopwords)
  - Stemming (PorterStemmer)
  - Lemmatization (WordNetLemmatizer)
- **intents.json**: A structured JSON file containing 13 intent objects. Each intent includes:
  - A unique tag
  - A list of patterns (user input examples)
  - A list of responses (including {location} placeholders and emoji)

Advanced Features Used

Several advanced features are implemented to enhance the functionality and realism of the chatbot:

- **Dynamic Placeholder Replacement**: The use of placeholders {name} and {location} in response templates allows for real-time personalization without hardcoding variations. This dynamic substitution gives the impression of a memory-aware conversational agent.
- **Emoji Integration**: Carefully chosen emojis are used throughout responses to:
  - Reinforce tone (e.g., 🌧️ for rain, 🌤️ for pleasant weather)
  - Improve readability and visual engagement
  - Make interactions feel less robotic and more social
- **Timezone-Aware Greeting Logic**: By detecting the user's local hour using their specified timezone, the bot provides time-appropriate greetings—an enhancement that reflects awareness of user context and mimics natural human social norms.
- **Scalable JSON Intent Design**: The intents.json structure supports easy scaling. Developers can:
  - Add new intents with minimal effort
  - Extend existing tags with more patterns and responses
  - Introduce multi-turn interactions or slot-filling in future versions
- **Robust Preprocessing**: The dual-layered preprocessing, applying both stemming and lemmatization that maximizes intent match accuracy even when user inputs vary linguistically.